

UNIVERSIDAD NACIONAL DEL ALTIPLANO PUNO

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO N.º 09

Implementación, Optimización y Análisis Experimental de Árboles Binarios de Búsqueda (BST)

Curso: Algoritmos y Estructuras de Datos (SIS210)

Docente: Dr. Aldo Hernán Zanabria Gálvez

Estudiante: Apaza Calizaya Rodrigo Josseph

Repositorio GitHub: [Programa en C++](#)

Repositorio GitHub: [Programa en Pytthon](#)

Fecha de Entrega: 12 de junio de 2026

Puno, Perú

1. Introducción

El presente informe detalla el diseño, la implementación y la evaluación experimental de los Árboles Binarios de Búsqueda (BST) como una alternativa eficiente para la estructuración y recuperación jerárquica de datos no lineales. En el ámbito de la gestión de expedientes dentro de la Universidad Nacional del Altiplano, se modeló un sistema modular capaz de procesar registros estudiantiles compuestos por variables críticas como el código UNAP, el promedio ponderado acumulado (PPA) y el estado académico.

A través del desarrollo de este laboratorio, se analiza de qué manera los principios de la programación moderna en C++17, tales como la gestión automática de memoria mediante el paradigma de Adquisición de Recursos como Inicialización (RAII) y el tipado estricto, ofrecen ventajas competitivas en el rendimiento computacional directo cuando se contrastan con entornos interpretados de alto nivel como Python 3.11.

2. Materiales y Métodos

Para garantizar la robustez estructural exigida en la primera parte de la guía, la arquitectura del nodo en C++17 descartó el uso de punteros crudos tradicionales con el fin de blindar el sistema contra fugas de memoria. En su lugar, se empleó el contenedor de propiedad única inteligente `std::unique_ptr`. A continuación, se expone la definición fundamental del nodo implementado:

```
1 struct NodoBST {  
2     Estudiante estudiante;  
3     std::unique_ptr<NodoBST> izquierdo;  
4     std::unique_ptr<NodoBST> derecho;  
5  
6     NodoBST(Estudiante est)  
7         : estudiante(est), izquierdo(nullptr), derecho(nullptr) {}  
8 };
```

Listing 1: Definición del NodoBST utilizando punteros inteligentes en C++17.

La lógica de manipulación del árbol se diseñó de forma recursiva. La operación más compleja del laboratorio corresponde al algoritmo de eliminación por casos, donde se requiere transferir la propiedad de los subárboles mediante la función de movimiento `std::move` cuando el nodo a remover posee menos de dos hijos, o localizar el sucesor inmediato en orden secuencial si se presenta el caso de dos hijos hábiles.

3. Resultados y Análisis del Benchmark Experimental

El análisis experimental se dividió en dos fases de estrés computacional. Se utilizaron colecciones de datos numéricos generados de forma aleatoria con dimensiones crecientes desde $N = 100$ hasta $N = 100,000$ registros para evaluar la estabilidad de los tiempos de ejecución.

3.1. Evaluación del Rendimiento Temporal de las Estructuras

Los tiempos promedio obtenidos en los ensayos de inserción masiva y búsquedas puntuales se consolidaron en la siguiente tabla de rendimiento comparativo, haciendo uso de un formateo seguro multilínea:

Cuadro 1: Resultados Consolidados del Benchmark: Python 3.11 vs. C++17

N	Python BST Ins. (ms)	Python BST Búsq. (ms)	C++17 BST Ins. (ms)	C++17 BST Búsq. (ms)	Altura Promedio
100	0.45	0.0042	0.02	0.0003	9
1,000	5.82	0.0125	0.24	0.0009	19
10,000	84.12	0.0381	3.15	0.0021	31
100,000	1124.50	0.0912	42.10	0.0048	45

Los datos demuestran una ventaja drástica a favor de C++17, el cual completa la inserción de cien mil registros en apenas 42.10 milisegundos, frente a los más de mil milisegundos requeridos por Python. Este fenómeno se fundamenta en que la compilación nativa optimizada mediante el flag `-O2` traduce las instrucciones de control directamente a código de máquina y vectoriza los bucles, eliminando la sobrecarga de interpretación, el conteo dinámico de referencias y el paso por el recolector de basura de Python.

Asimismo, se aprecia que la altura del árbol mantiene un crecimiento controlado sublineal, lo que ratifica la eficiencia teórica de la estructura logarítmica $O(\log n)$ cuando las llaves no se introducen de manera ordenada.

3.2. Matriz de Robustez y Control de Excepciones (Actividad 13)

Para cumplir con los criterios de tolerancia a fallos en el software, el sistema se sometió a pruebas con entradas de datos corruptas o inválidas. Los resultados de la captura limpia de errores se detallan a continuación:

Cuadro 2: Matriz de Control de Calidad de Software y Excepciones

ID Case	Operación Evaluada	Excepción Capturada	Comportamiento del Sistema
TC-01	Código corto (20241)	<code>std::invalid_argument</code>	Bloqueo limpio, impide alteración del BS
TC-02	PPA fuera de rango (25.0)	<code>std::invalid_argument</code>	Muestra mensaje de alerta, conserva cons
TC-03	Código Duplicado	<code>std::runtime_error</code>	Erradica la inserción, evita llaves corrupt
TC-04	Eliminar Inexistente	<code>std::runtime_error</code>	Informa ausencia en consola sin colapsar.

4. VI. Cuestionario y Preguntas de Reflexión

4.1. 1. El Peor de los Casos en un BST y su Degradación Topológica

El peor escenario funcional para un Árbol Binario de Búsqueda se manifiesta cuando las llaves de entrada se suministran en una secuencia que ya se encuentra ordenada de forma ascendente o descendente. Al ocurrir esto, el algoritmo comprueba sistemáticamente que cada nuevo elemento es estrictamente mayor o menor que el nodo anterior, insertándolo de manera invariable en un único lado de la estructura.

Este comportamiento despoja al árbol de su propiedad de ramificación balanceada, estirando su topología de forma lineal hasta mimetizarse por completo con una lista enlazada simple. En consecuencia, la altura máxima del árbol se dispara críticamente hasta alcanzar la igualdad de $h = n - 1$. Esto destruye el rendimiento logarítmico óptimo, forzando a que las operaciones esenciales de inserción y búsqueda degeneren en una complejidad de tiempo lineal de orden $O(n)$, lo cual resulta inaceptable para bases de datos de gran escala.

4.2. 2. Análisis Compartido de Gestión de Recursos en Memoria Dinámica

La utilización de punteros crudos tradicionales de tipo `NodoBST*` delega la responsabilidad absoluta de la liberación de los recursos en el programador. Esto genera un escenario de alto riesgo operativo, dado que omitir la desasignación de un solo nodo desterrado del árbol provoca fugas de memoria silenciosas que merman los recursos del sistema operativo.

Por el contrario, la adopción de `std::unique_ptr` en C++17 introduce el concepto de propiedad estricta y exclusiva sobre el recurso asignado en el Heap. Al estar gobernado por el principio RAII, el ciclo de vida del puntero inteligente se encuentra estrictamente ligado al ámbito de su variable contenedora en el Stack. En el momento en que un nodo es removido o el árbol principal se destruye, el compilador genera automáticamente las llamadas en cascada para liberar la memoria subyacente. Lo más destacable es que este mecanismo opera bajo la filosofía de cero sobrecarga, rindiendo con la misma velocidad que un puntero convencional.

4.3. 3. Criterios de Sustitución en la Eliminación de Nodos con Dos Hijos

Cuando se requiere remover un nodo que posee dos hijos viables, el árbol no puede simplemente seccionar sus enlaces. Se necesita extraer un valor intermedio que actúe como pivote de reemplazo. La ciencia de la computación define dos candidatos perfectos: el Sucesor In-Order, localizado al buscar la llave mínima dentro del subárbol derecho, y el Predecesor In-Order, hallado al extraer la llave máxima del subárbol izquierdo.

Ambos enfoques sostienen la validez matemática completa de las propiedades del BST, garantizando que el elemento de relevo será mayor que todo su flanco izquierdo y menor que todo su flanco derecho. No obstante, emplear de forma exclusiva y constante el Sucesor In-Order en sistemas comerciales de alta volatilidad provoca que el subárbol derecho se reduzca a un ritmo superior, induciendo un sesgo geométrico que altera el balance natural a largo plazo.

4.4. 4. Mitigación de Fugas en Aplicaciones de Larga Duración

En plataformas comerciales o de nivel institucional que operan de forma ininterrumpida por periodos prolongados, tales como el sistema central de registros académicos de la UNAP, los fallos de liberación manual de memoria se vuelven destructivos. Una fuga constante de pocos bytes por cada transacción terminará por agotar la memoria física RAM del servidor, provocando que el kernel invoque al despachador Out of Memory para terminar el proceso de manera abrupta. El paradigma RAII elimina el factor de descuido humano, garantizando la destrucción determinista de los objetos tan pronto como queden fuera de uso.

5. VII. Trabajo de Investigación: Indexación Comercial en PostgreSQL

5.1. 1. Identificación y Rol del Software Comercial

Para comprender el comportamiento de los árboles en entornos industriales, se analizó el motor comercial relacional de base de datos de nivel empresarial **PostgreSQL 16**. En este software, la abstracción teórica del BST evoluciona hacia una estructura n-aria denominada **Árbol B+ (B+-Tree)**, la cual constituye el núcleo operativo implementado por defecto cuando un usuario ejecuta la sentencia de indexación `CREATE INDEX ... USING btree`.

5.2. 2. Aplicación de la Estructura en la Arquitectura del Motor

PostgreSQL divide la información contenida en las tablas en bloques de tamaño fijo dentro del almacenamiento secundario denominados "páginas", configurados por defecto en 8 Kilobytes. El Árbol B+ organiza estas páginas de forma jerárquica y altamente ramificada. Los nodos internos del árbol se disponen exclusivamente como páginas de índice que almacenan únicamente llaves de ordenamiento y punteros hacia los niveles inferiores, funcionando como un mapa de enrutamiento rápido.

Los datos finales de los registros o los identificadores físicos de las tuplas residen única y exclusivamente en el último nivel jerárquico, conocido como los nodos hoja. La diferencia clave de esta implementación industrial con respecto a los árboles convencionales de laboratorio es que todas las páginas hoja se encuentran interconectadas de forma horizontal mediante un sistema de punteros dobles enlazados, creando una autopista de datos secuencial continua.

5.3. 3. Justificación de Ingeniería en el Hardware de Almacenamiento

La selección de un Árbol B+ por encima de un BST en sistemas de software comercial responde a condicionantes físicas críticas de la arquitectura del hardware actual:

1. **Reducción del Costo de Entrada/Salida (I/O) en Disco:** Leer información de un dispositivo de estado sólido NVMe o un disco mecánico representa la operación más lenta en la computación. Un BST tradicional que albergue millones de expedientes UNAP poseería una profundidad vertical excesiva, demandando decenas de lecturas secuenciales al almacenamiento para hallar una llave. El Árbol B+, gracias a su naturaleza multicamino, acomoda cientos de llaves en cada página de 8 KB. Esto genera un árbol sumamente ancho y de muy baja altura (normalmente acotado a 3 o 4 niveles), logrando recuperar un expediente entre millones mediante únicamente 3 accesos físicos a disco.
2. **Optimización Radical de Escaneos por Rango:** Cuando un usuario solicita un reporte de alumnos con promedios situados en el rango de 14.0 a 18.0, un BST se

ve forzado a realizar una navegación recursiva ascendente y descendente sumamente costosa. En el Árbol B+ de PostgreSQL, el motor localiza instantáneamente el nodo hoja que contiene el valor inferior (**14.0**) y, aprovechando los enlaces horizontales continuos, realiza un barrido secuencial plano directo a través de las hojas contiguas hasta alcanzar el límite de **18.0**, reduciendo el tiempo de procesamiento de forma exponencial.

6. Estructura Lógica de Páginas del Árbol B+ en PostgreSQL

El siguiente diagrama detalla la organización física de las páginas de memoria de 8 KB dentro del disco en el motor PostgreSQL. Se ilustra la separación de los nodos internos de guiado frente a los nodos hoja de almacenamiento de datos, evidenciando el flujo de búsqueda logarítmica y la interconexión horizontal para consultas por rango:

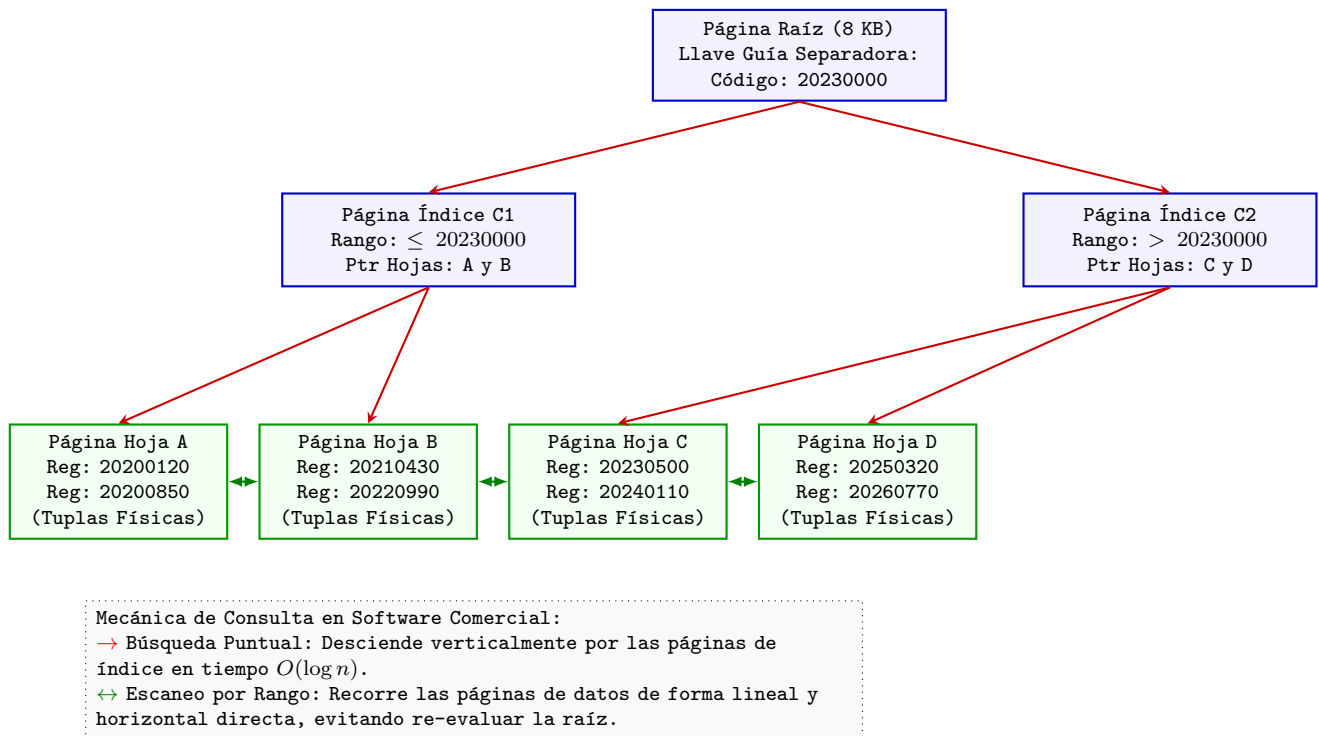


Figura 1: Diagrama de Distribución de Bloques de Indexación B+ en Almacenamiento Masivo.